Module-1 : Psendocodes :-

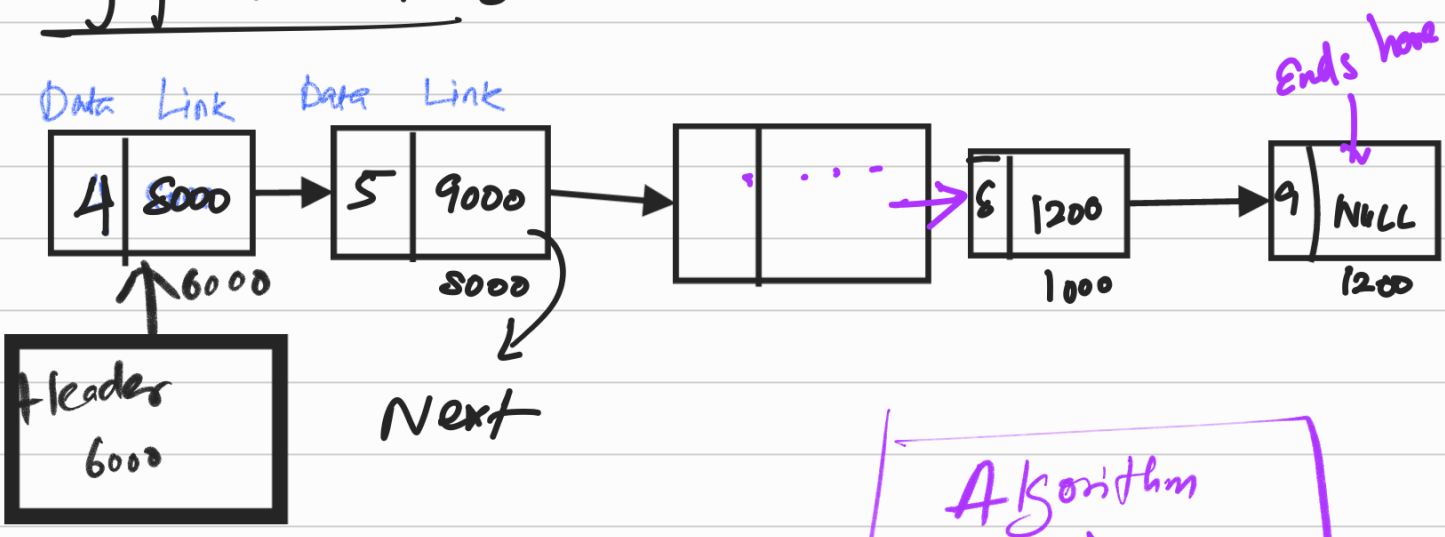
Singly linked list → Insertion @ Beginning & at Ending.

Psendocode :

```
Struct Node {
        int Data;
        Node * next;
}
void insertAtBeginning(head, data) {
  // create node.
  NewNode = create NewNode(value);
   NewNode.next = head;
      head = NewNode;
}
void insertAtEnding(head, data) {
  // create Node.
 NewNode = create NewNode(value);
   if(head !== NULL) {
```

```
head = newNode;
}
else {
    temp = head
    temp.next = temp + 1
    temp = NewNode;
}
```

Singly-linked list :



DSAA → Program = Algorithm + Data Structure

Primitive

Int, float, String, double ...

Non-Primitive

Linear

Array, List, Stack, Queue

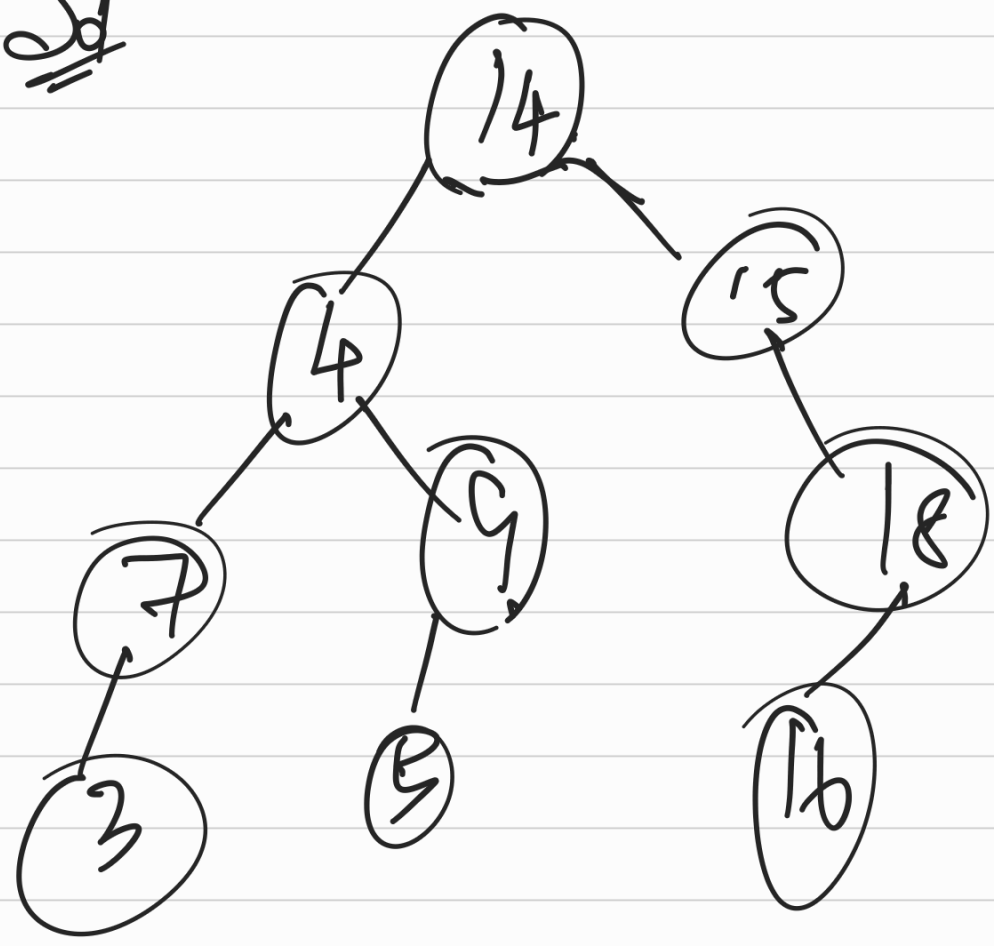
Non-Linear

Trees, Graphs

Binary Search Tree :

14, 15, 4, 9, 7, 18, 3, 5, 16.

Sol



Complexity Analysis :

- Best
- Avg
- Worst

1. Time
2. Space
3. Asymptotic

Big OH

$f(n) \leq g(n)$

- Includes Worst-Case
- Frequent use

Omega

$f(n) \geq g(n)$

Best Case

Theta (θ)

Avg. Case

$c_1 g(n) \leq f(n) \leq c_2 g(n)$

Little OH $\approx$ Big OH

- Little OH → Upper Bound.
- Big OH → Lowest-Upper Bound

Little OH

$f(n) < g(n)$

Module - 4

Module - 4

Bubble Sort : → Bigger elements move to end.

* Always move the Bubble right.
*

8	1	18	12	9	4	22

Step-1 : Check 8 & 1 :

As 8 > 1 ; Bubble = 8

and move right.

1	8	18	12	9	4	22

Step-2 : Check 8 and 18 :

As 8 < 18 ; Bubble = 18 ;

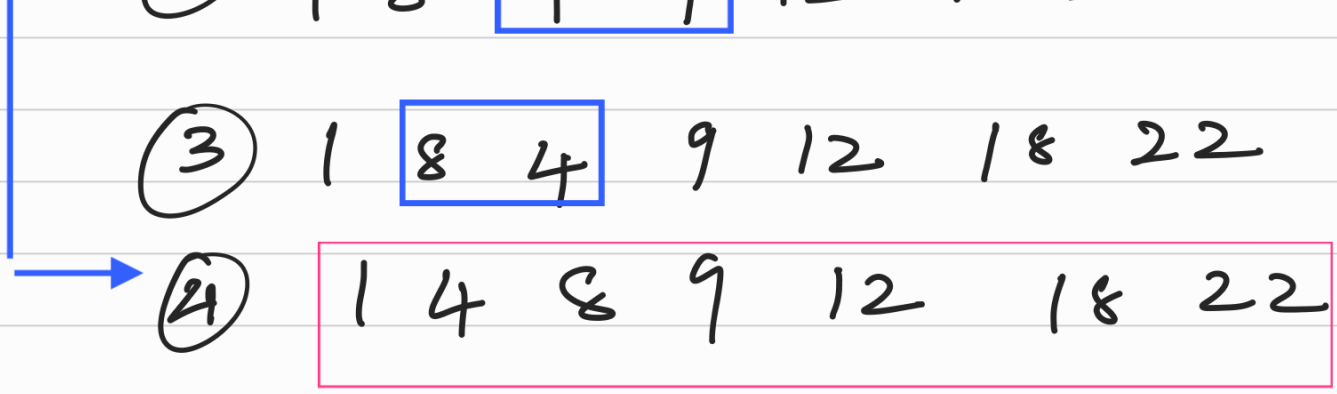
then go on to next

1	8	18	12	9	4	22

Similarly, perform all these steps until all the nodes are achieved.

Answer : ① 1 8 | 12 9 | 4 18 22

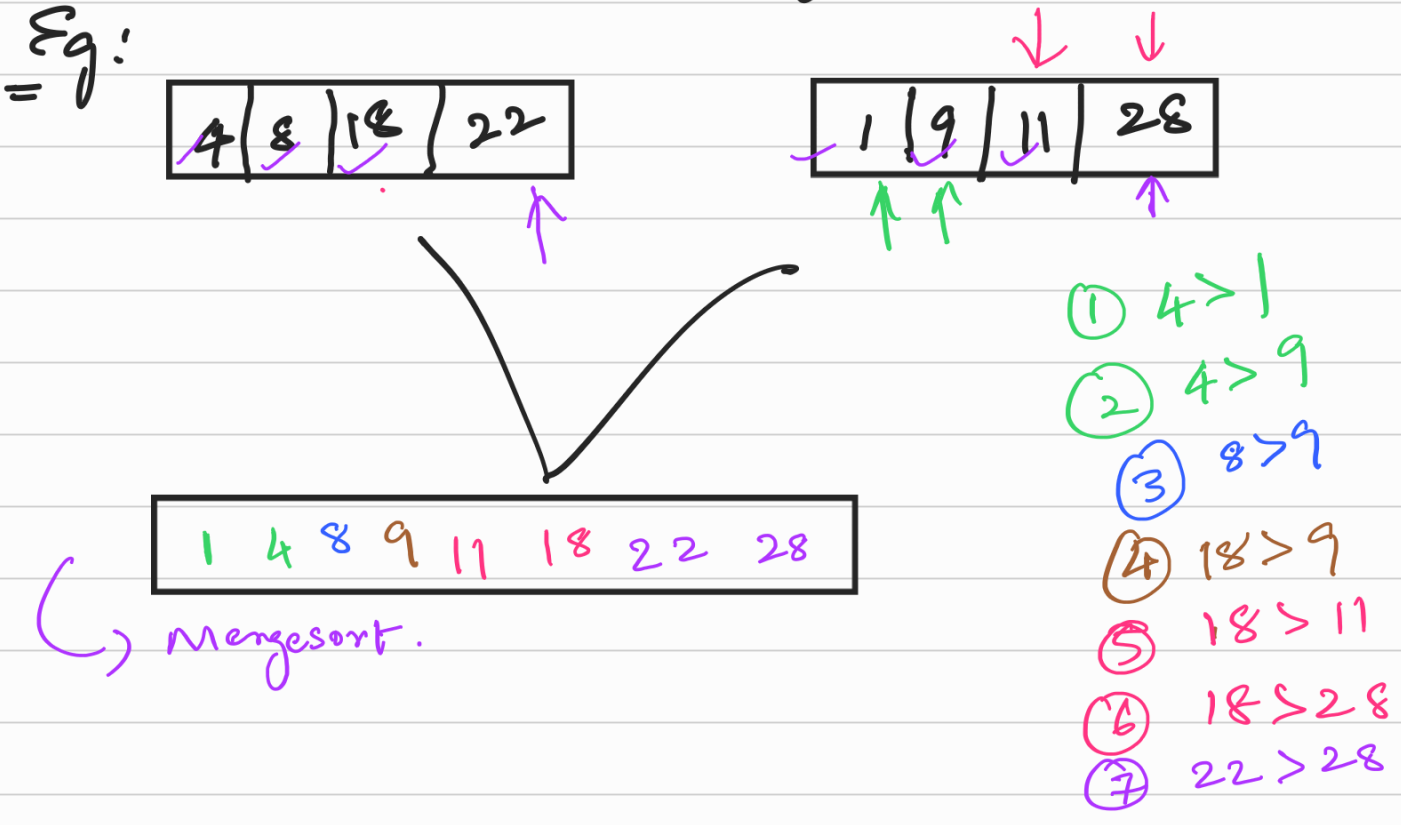
② 1 8 | 9 4 | 12 18 22



MergeSort :

* Involves merging & sorting of two arrays.

Eg:

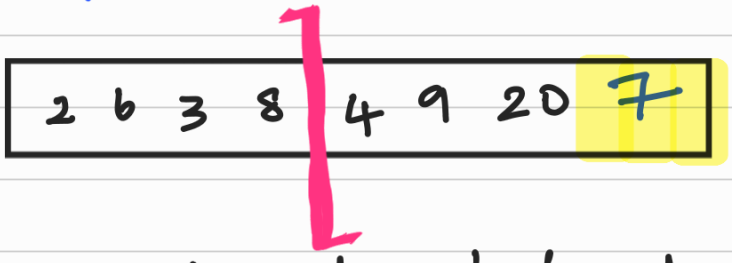


Quicksort :

* Grab a pivot element.
  Set last element = pivot

Pivot = 7

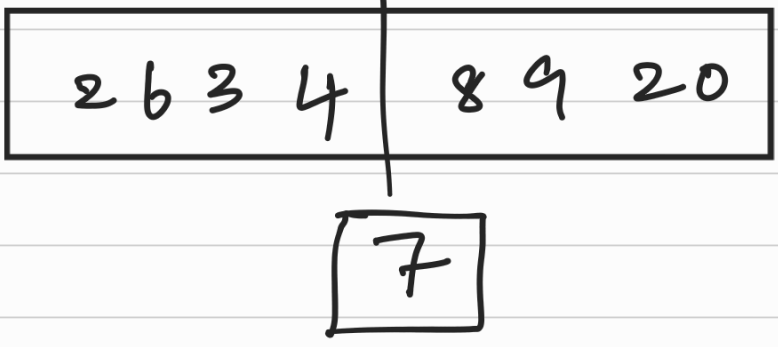
* Create a wall.



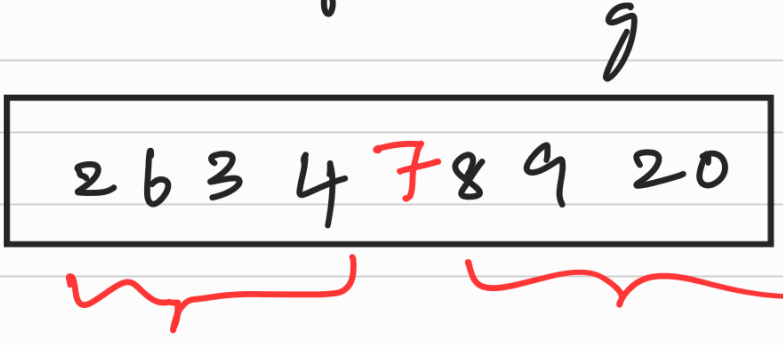
if val < pivot keep it Left.

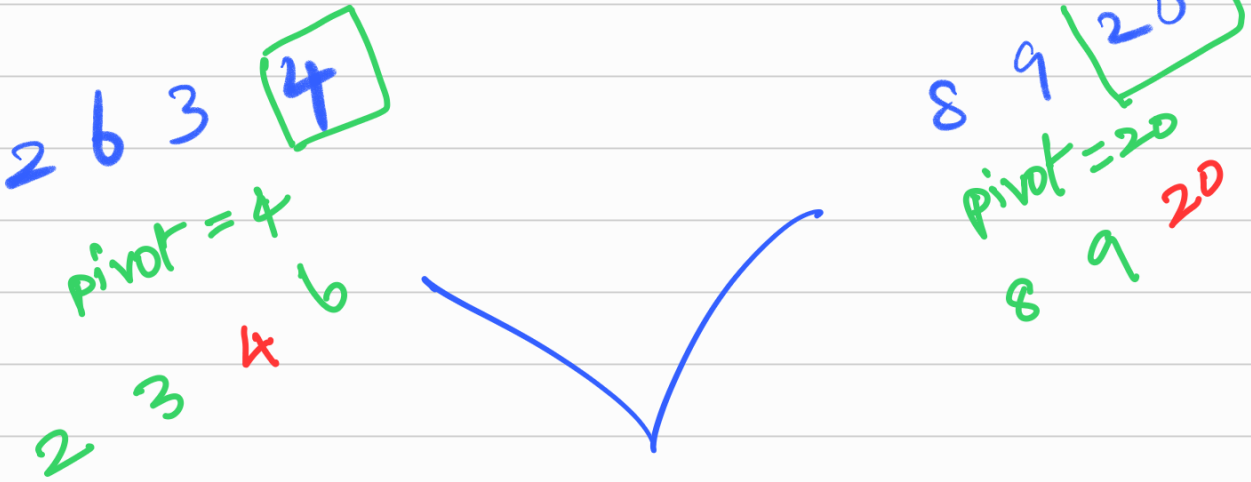
else :

move right



fix





Merging all :

∴ Quicksort is done.

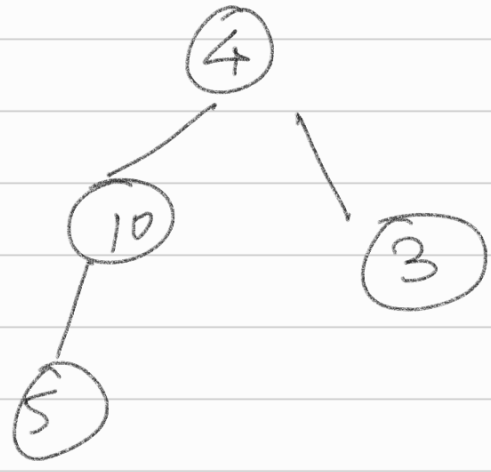
Heap Sort:

* Improved version of selection sort.
* Heapsort is faster when time complexity of quicksort is worst.

(largest element is selected & set to the last position.)

4	10	3	5	1
0	1	2	3	4

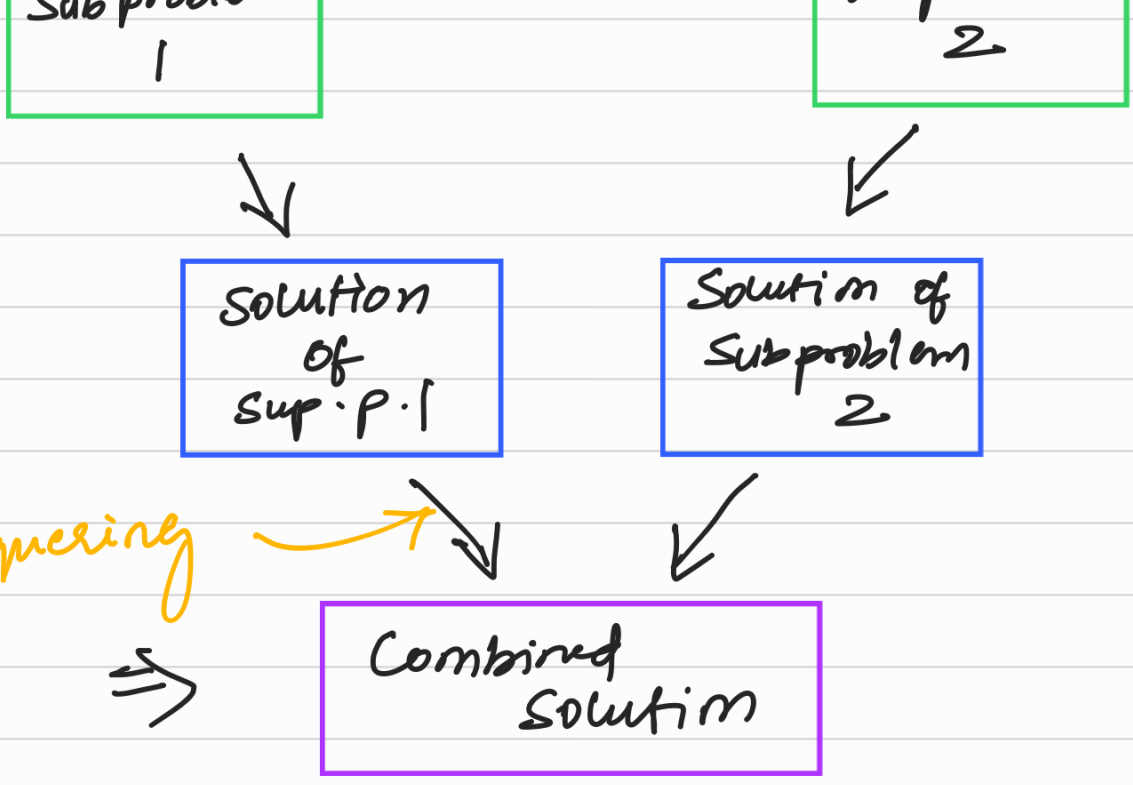
Build Heap:-



Radix Sort:

* Refer Onenote.

Divide & Conquer:



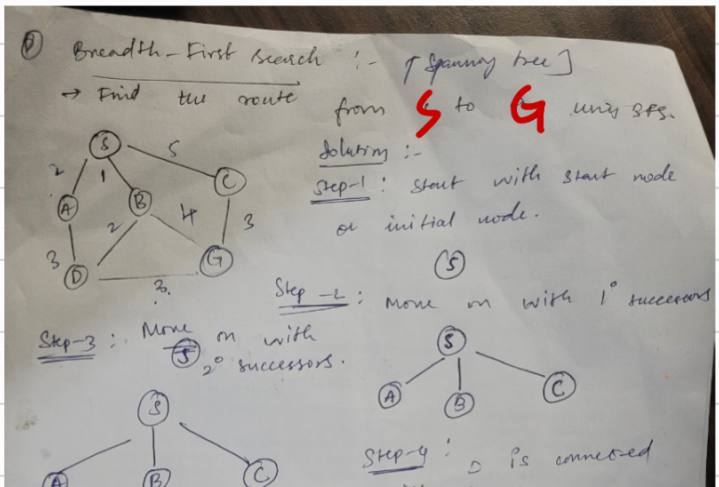
Design Methods :

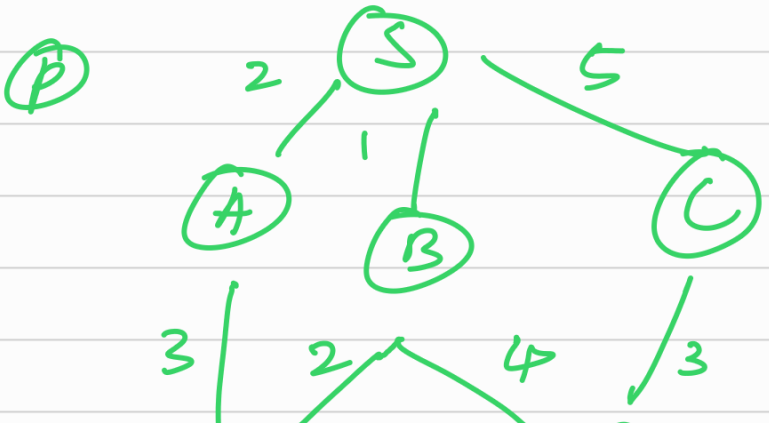
* Divide & Conquer
* Greedy Paradigm
* Dynamic program paradigm.

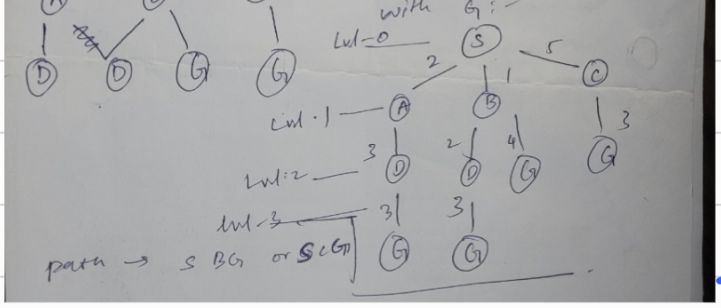
→ Backtracking Algorithms.

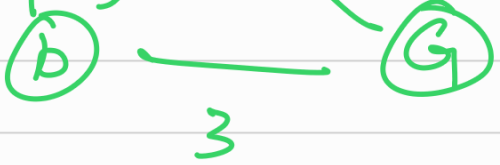
→ Overlapping Subproblems.

Breadth First Search [BFS]







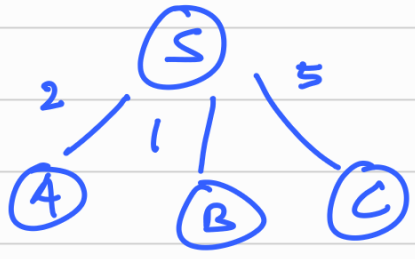


3

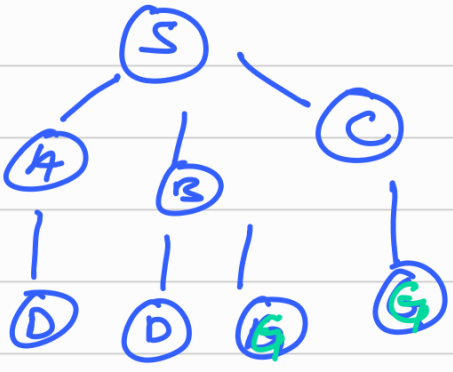
Sol

Start node = S
End node = G

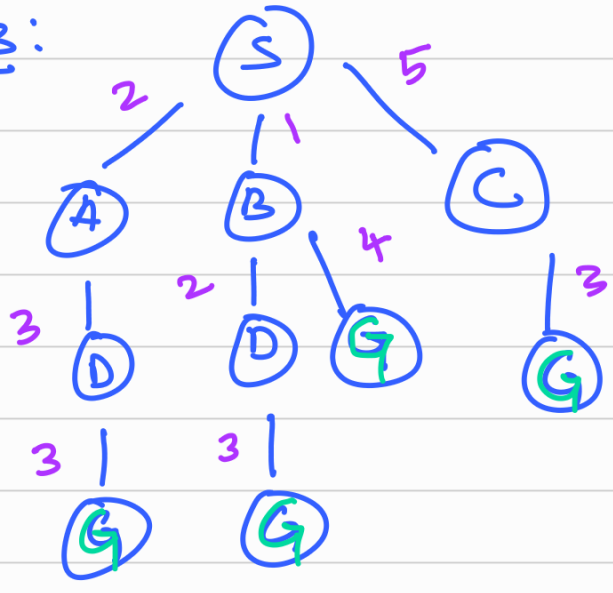
Step-1 : 1° Successors :-



Step-2 : 2° successors



Step-3 :



Step-4: Now, find the shortest path to reach the goal node.

$S \to A \to D \to G$

$= 2 + 3 + 3$

$= 8$

$S \to B \to D \to G$

$= 1 + 2 + 3$

$= 6$

$S \to B \to G$

$= 1 + 4$

$= \boxed{5}$ → shortest distance

$S \to C \to G$

$= 5 + 3$

$= 8$

$S \to B \to D \to G$

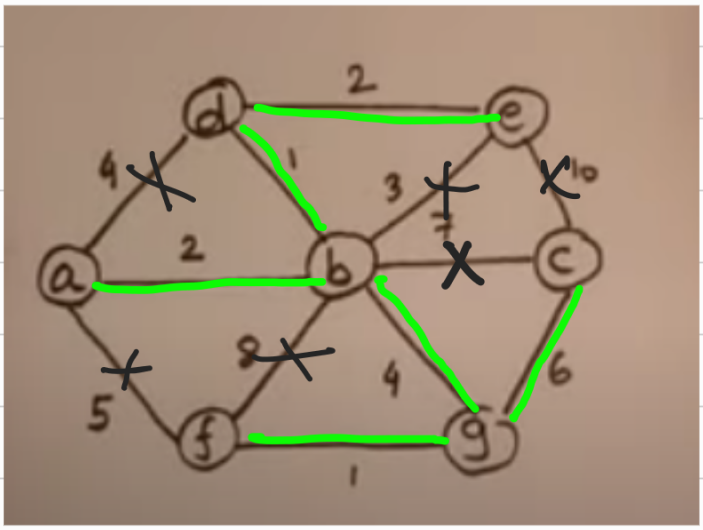
$= 1+2+3$

$= 6$

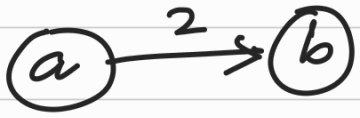
∴ According to Breadth First Search [BFS] the optimal path is S → B → G

Module-5 :-

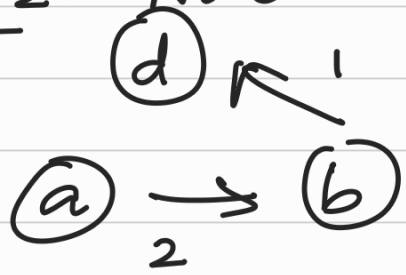
Prim's Algorithm:



Step-1 : Node A → B

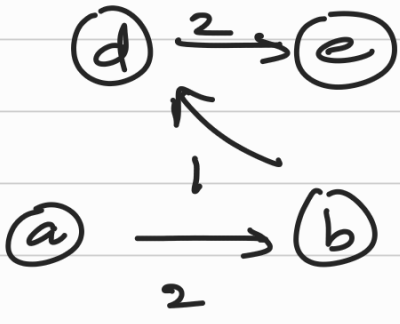


Step-2 : Node B → d

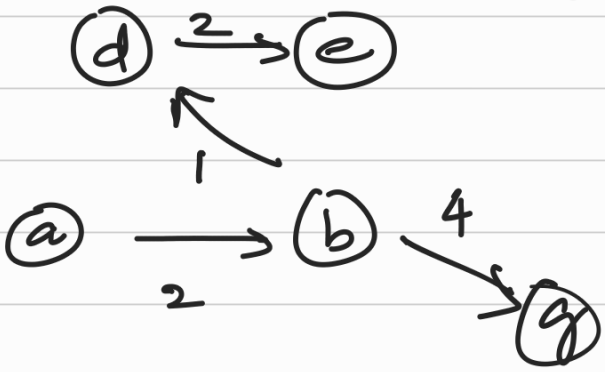


* All vertices should be connected.
* The resultant should not have any cycles.
  NO CYCLES

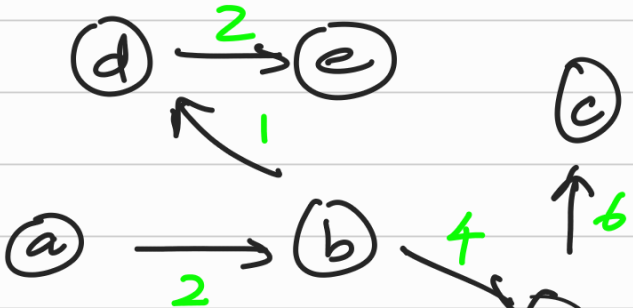
Step-3 : Node D → e



Step-4 : Node b → g



Step-5 : g → c & g → f



∵ weight of this spanning tree
= 2 + 1 + 2 + 4 + 6 + 1

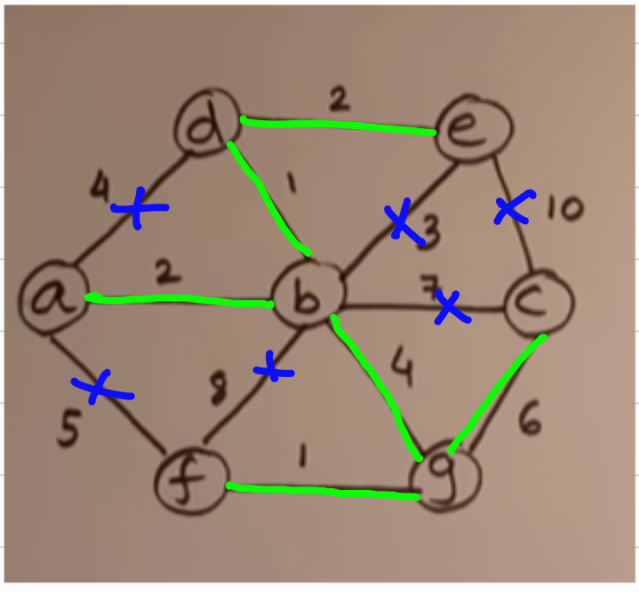
wt = 16

Kruskal's Algorithm :

https://youtu.be/71UQH7Pr9kU

* Start from the smallest edge.

* keep including min. edges,
Such that no cycles are formed.



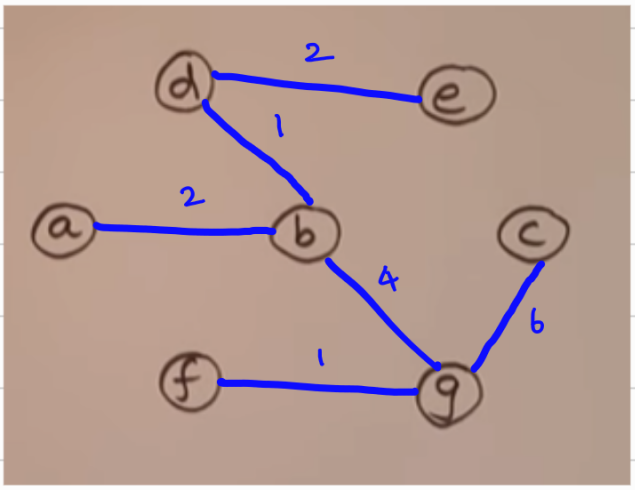
x = forms a cycle

→ **lowest edge = 1.**

then move to 2 ...
then 3 ...

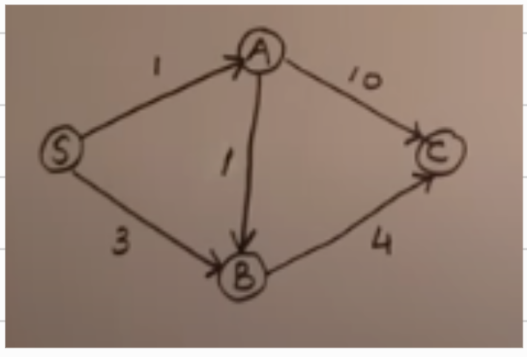
Weight of tree = 2 + 1 + 2 + 1 + 4 + 6

wt = 16



Djikstra's Algorithm:

* To find the shortest path.
* Categorise the vertices into Explored & unexplored vertices.
* D = min distance.
  P = parent.



Explored	Unexplored
S^0	$S^0\ A^\infty\ B^\infty\ C^\infty$
S A	$A^{15}\ B^{35}\ C^\infty$
S A B	$B^{21}\ C^{11A}$
S A B C	$C^{6\,B}$

Path:

$S \rightarrow A \rightarrow B \rightarrow C$

$S \rightarrow A$: 1

$S \rightarrow A \rightarrow B$: $1 + 1 = 2$

$S \rightarrow A \rightarrow B \rightarrow C$: $1 + 1 + 4 = 6$

∴ Shortest path = $S \rightarrow A \rightarrow B \rightarrow C$

Distance = $1 + 2 + 6 = 9$ units.

Time complexity:

4 - categories → Asymptotic Notation: Representation of Time complexity in Mathematical form.

Big-oh	Omega	Theta	Little-oh

Big-OH → Includes Worst-case

$f(n) \le c \cdot g(n)$

Omega (Ω) → Includes Best-case

$f(n) \ge c \cdot g(n)$

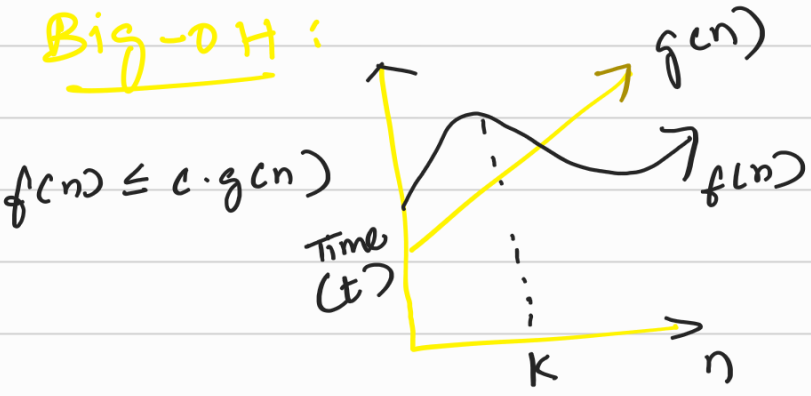
Theta (Θ) → Includes Avg. case

$c_1 g(n) \le f(n) \le c_2 g(n)$

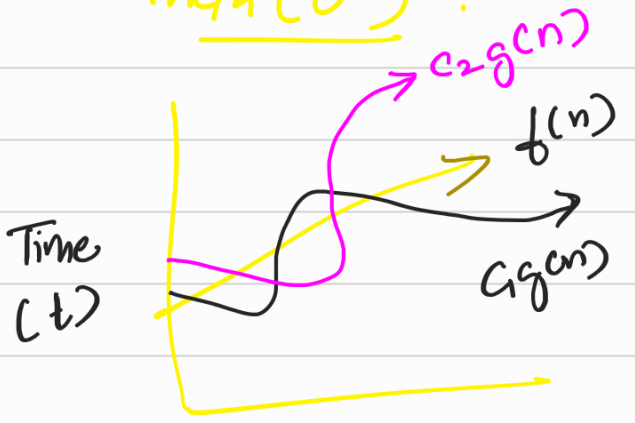
Little-oh → Same as Big-OH (upper Bound)

$f(n) < c \cdot g(n)$

Big-OH :

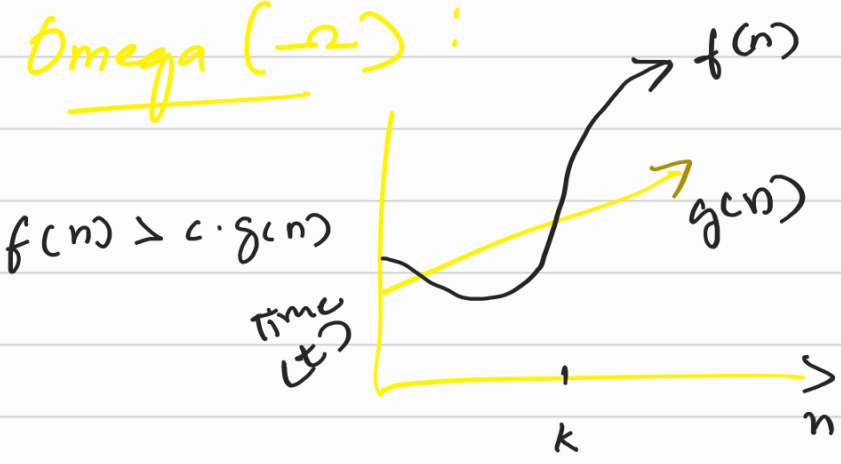


Theta (Θ) :



$c_1 g(n) \le f(n) \le c_2 g(n)$

Omega (Ω) :



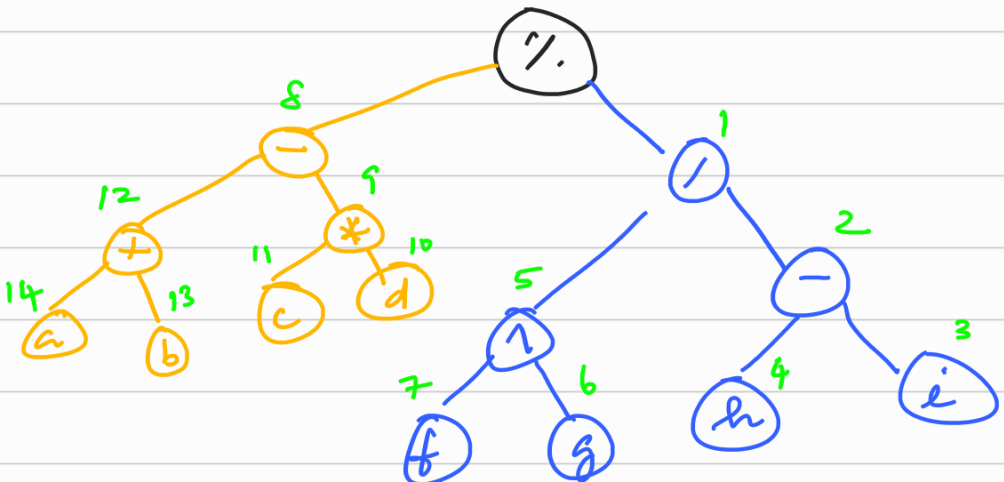
(P) ab + cd * − fg ∧ h i − / %

To do: Conversion to postfix form:

Sol

Always Start from right.

ab + cd * − fg ∧ h i − / %.



Solution: $[(a+b)-(c*d)] \,\%\, [(f \wedge g)/(h-i)]$

Objective (High Value Target):-

Module-1:

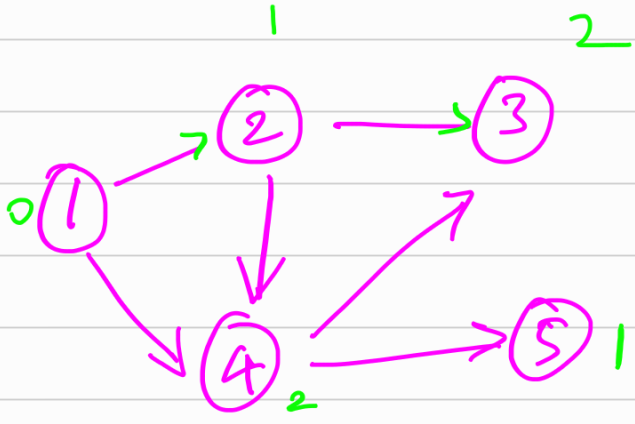
* Start with order 0.

→ Topological sort:

* Should be a DAG

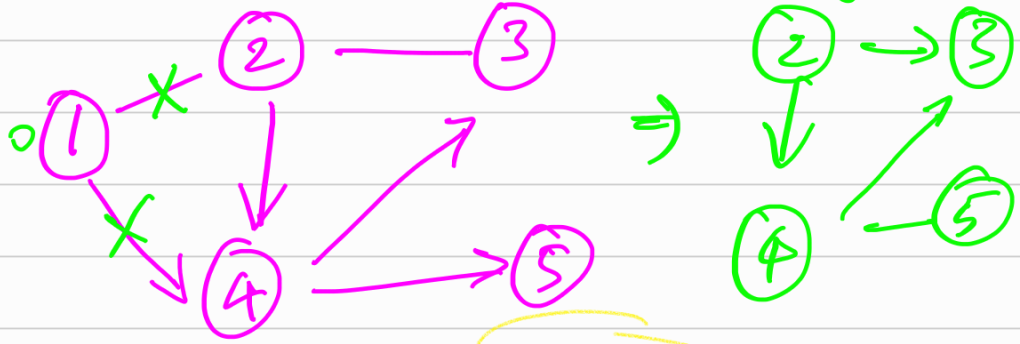
DAG: (Directed Acyclic Graph)

* Must not contain any cycles; should be directed.
* Edge $u \to v$: u must come before v, for topological order.



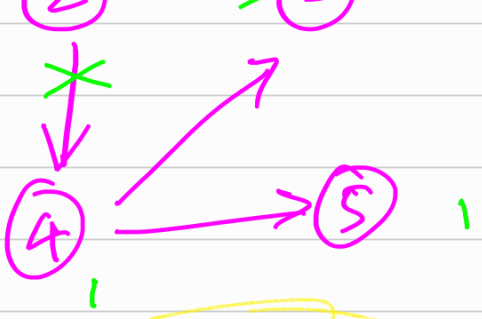
Sol

Step-1: Start with node order 0 @ Node 1



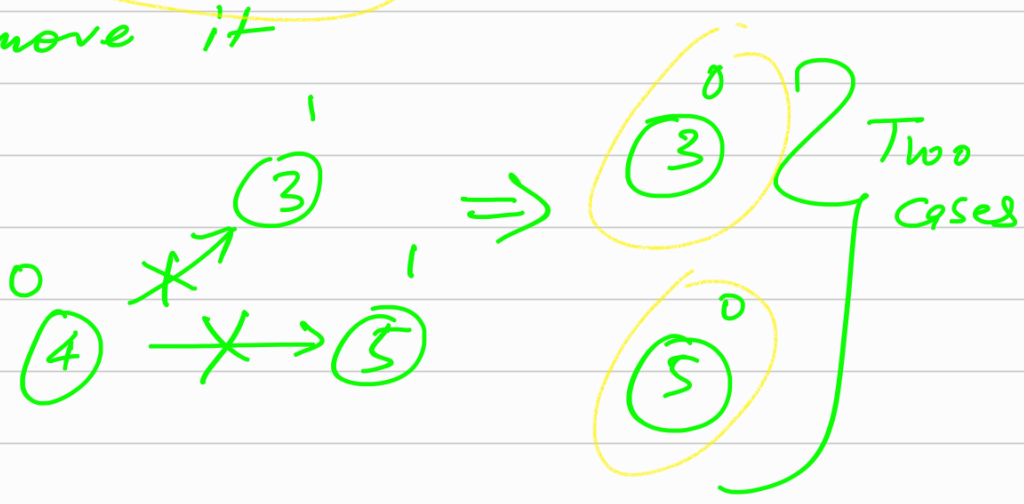
Step-2: Start @ Node 2

node = 2.



Step-3: Node 4:

Remove it



FINAL: Two cases are available

Case-I: 1 2 4 3 5

Case-II: 1 2 4 5 3

Vijayajeyakumar